

نسخه رسمی

پروژه رادار (Radar)

معماری فنی و مهندسی

مرجع فنی جامع و راهنمای پیاده سازی سامانه رادار

مستندات معماری، ساختار داده ها، خط تریاژ سه لایه، احراز هویت چندمستأجری و اسکنر پس زمینه خودکار

سطح سند:

رسمی و اختصاصی

تاریخ بازبینی:

بهمن ۱۴۰۳ / اکتبر ۲۰۲۶

محصول:

رادار (موتور کشف نیت خرید B2B)

Radar — Technical Reference (مرجع فنی جامع)

`README.md` Persian body, English headings and technical terms. Companion to

.and `docs/TechnicalDocumentation.md` (design philosophy) (user guide)

.Last updated for the webhook-ingestion + 3-layer-pipeline release

Table of Contents |

Overview .1	•
System Architecture .2	•
Tech Stack .3	•
Data Model .4	•
Triage Pipeline .5	•
Ingestion .6	•
API Reference .7	•
Radar Assistant .8	•
Authentication & Security .9	•
Configuration .10	•
Frontend & Realtime .11	•
Deployment .12	•
Operations Runbook .13	•
Project Structure .14	•
Known Limitations & Roadmap .15	•

Overview.1 — نمای کلی

رادار (Radar) یک موتور محلی کشف نیت خرید (Buying-Intent Engine) برای تیم‌های فروش فعال در ایران است. پیام‌های جوامع فارسی (تلگرام، بله، ایکس، فروم‌ها) را می‌خواند، نویز را با سه لایه فیلتر حذف می‌کند، به هر پیام امتیاز ۰ تا ۱۰۰ می‌دهد و پیش‌نویس پاسخ فارسی آماده ارسال تولید می‌کند — همه روی یک سرور، بدون وابستگی به کلاود خارجی.

اعداد کلیدی پیاده‌سازی فعلی:

شاخص	مقدار
لایه‌های تریاژ	۳ (قطعی، غربال سریع، تحلیل عمیق)
سطوح نیت	irrelevant ، curious ، problem_aware ، high_intent
وضعیت پیام خام	error ، filtered ، processed ، pending
وضعیت سرنخ	dismissed ، contacted ، approved ، new
پلتفرم‌های منبع	forum ، twitter_x ، bale ، telegram
هزینه لایه ۰ و ۱	صفر توکن (CPU خالص)
تایم‌اوت فراخوانی LLM	۱۲ ثانیه، سپس fallback

نمودار معماری و جریان داده

```
graph TD
    A[Iranian Communities<br/>Telegram · Bale · X · Forums] -->|bot webhooks| W[Ingest API<br/>/api/ingest]
    W -->|pending| B[(raw_messages<br/>PocketBase / SQLite)]
    A -->|manual / demo| B
    B -->|pending queue| C[3-Layer Triage Pipeline<br/>lib/pipeline.ts]
    C -->|L0 reject: 0 tokens| F
    C -->|L1 unqualified: 0 tokens| F
    C -->|L2 prompt| D[LLM Gateway<br/>Ollama / vLLM / Domestic Proxy]
    D -->|structured JSON| C
    D -->|offline fallback| E[Persian Heuristic Engine]
    E --> C
    C -->|score + cost| F[(leads<br/>PocketBase / SQLite)]
    F -->|SSE subscription| G[Next.js 16 Dashboard<br/>app/page.tsx + app/dashboard]
    G -->|1-click draft| H[Sales & Marketing Team]
```

مسیر داده همیشه یک طرفه است: **communities → raw_messages → leads → dashboard**.

هیچ مسیری از داشبورد به پلتفرم‌ها (ارسال خودکار پیام) وجود ندارد؛ ارسال پاسخ

همیشه با تصمیم انسانی انجام می‌شود (Human-in-the-loop).

نقش اجزا:

جزء	فایل	مسئولیت
Ingest API	<code>app/api/ingest/*/route.ts</code> <code>lib/ingest.ts</code>	دریافت وب‌هوک، dedupe، ساخت source ، درج pending
Triage Pipeline	<code>lib/pipeline.ts</code>	اجرای L0/L1/L2 و ثبت lead — تنها نقطه تریاژ
Layer 0/1	<code>lib/prefilter.ts</code>	قوانین قطعی و غربال سریع، بدون توکن
Layer 2	<code>lib/llm.ts</code> (<code>evaluateMessageWithLLM</code>)	پرامپت کامل + fallback 启发式
Worker	<code>scripts/worker.ts</code>	مصرف صف pending (one-shot)؛ با timer تکرار می‌شود)
Schema bootstrap	<code>pocketbase/setup_schema.ts</code>	ساخت کالکشن‌ها + مهاجرت‌ها + seed محصول و منابع
Dashboard	<code>app/dashboard/page.tsx</code> , <code>app/page.tsx</code>	فید زنده روی SSE ساب‌اسکریپشن PocketBase

لایه	فناوری
Runtime / Package manager	+Bun 1.4
Framework	Next.js 16 (App Router, Turbopack) · React 19
Language	TypeScript 5 (strict)
Styling	<code>app/globals.css</code> + Tailwind CSS v4
Database & realtime	PocketBase (embedded SQLite) + SSE subscriptions
AI inference	OpenAI-compatible endpoint (Ollama/vLLM/domestic gateway) + heuristic fallback
Ingestion protocols	(<code>tapi.bale.ai</code>) Telegram Bot API webhooks · Bale Bot API webhooks
Typography	(بدون CDN خارجی) Ravi + Estedad, self-hosted
Icons	Inline SVG در <code>components/Icons.tsx</code> (بدون پکیج آیکون)

قواعد معماری (از `AGENTS.md`): اول TypeScript خالص (بدون یوتیلیتی اضافه)، صفر
CDN خارجی، رابط کاملاً RTL با اعداد فارسی.

چهار کالکشن در PocketBase. نوشتن مستقیم (create/update/delete) در همه

کالکشن‌های داده فقط برای superuser قابل نوشتن هستند (`createRule/updateRule/deleteRule = null`)؛

خواندن (list/view) برای داده‌های عمومی باز است. دسترسی رکوردهای کاربری در کالکشن `users`

منحصراً به خود کاربر محدود است (`id = @request.auth.id`). تمام تغییرات سیستمی از

سمت سرور با احراز superuser یا احراز کاربر انجام می‌شود.

`users` — کاربران و هویت سازمانی (کالکشن Auth)

فیلد	نوع	توضیح
<code>email</code>	email (required, unique)	ایمیل سازمانی / ورود
<code>name</code>	text (required)	نام کاربر
<code>company</code>	text	نام شرکت یا سازمان
<code>role</code>	text	سمت سازمانی (مدیر فروش، بنیان‌گذار و ...)
<code>product_name</code>	text	نام تجاری محصول تحت پایش
<code>product_description</code>	text	شرح ارزش‌آفرینی محصول
<code>ideal_customer_profile</code>	text	پرسونای مشتری ایده‌آل (ICP)
<code>onboarding_completed</code>	bool	وضعیت تکمیل موفق فرآیند آنبوردینگ

فیلد	نوع	توضیح
name	text (required)	نام محصول
tagline	text	تگ لاین
description	text (required)	شرح — وارد پرامپت L2 می شود
value_propositions	json	مزیت ها — وارد پرامپت L2 می شود
ideal_customer_profile	text (required)	ICP — وارد پرامپت L2 می شود
keywords	json	کلیدواژه های پایش — ورودی غربال L1

فیلد	نوع	توضیح
name	text (required)	مثل Telegram: Group Title (-100123) — وب هوک ها خودکار می سازند
platform	select	`telegram` `bale` `twitter_x` `forum`
status	select	`active` `paused`

فیلد	نوع	توضیح
source_id	→ relation sources	منبع (expand می شود تا پلتفرم معلوم شود)
external_id	text	شناسه یکتا در سطح پلتفرم (tg::) — کلید dedupe
author_handle	text (required)	مثل @username
content	text (required)	متن پیام (حداکثر ۳۰۰۰ کاراکتر در ingest)
thread_context	text	کانتکست گفت وگو
posted_at	date	زمان ارسال در پلتفرم مبدأ
status	select	`pending` → `processed` `filtered` `error`

فیلد	نوع	توضیح
raw_message_id	relation → raw_messages (required)	پیام مبدأ
product_id	relation → products (required)	محصول مبنای ارزیابی
intent_score	number 0-100 (required)	امتیاز نیت
intent_level	select (required)	چهار سطح نیت
reasoning	text	استدلال فارسی
matched_feature	text	ماژول منطبق محصول (فارسی)
suggested_reply	text	پیش نویس پاسخ فارسی (برای irrelevant خالی)
input_tokens / output_tokens	number	حسابداری توکن (برای L0/L1 صفر)
estimated_cost_usd	number	هزینه دلاری (برای L0/L1 صفر)
lead_status	select	چرخه حیات: new → approved/contacted/dismissed

رابطه‌ها: SOURCES ||--o{ RAW_MESSAGES ، RAW_MESSAGES ||--o{ LEADS ، PRODUCTS ||--o{ LEADS

5. Triage Pipeline — خط تریاژ سه لایه

نقطه ورود واحد: `trriagePendingMessage(pb, rawMsg, product)` در

`lib/pipeline.ts`. همه مصرف کننده ها (`worker`)، `/api/analyze`،

`/api/simulate`، هر دو وب هوک) از همین تابع استفاده می کنند تا رفتارها واگرا

نشوند. برای متن های ثبت نشده (`ad-hoc`) نسخه بدون `persistence` با نام

`trriageAdHocMessage` وجود دارد.

Layer 0 — فیلتر قطعی (`lib/prefilter.ts::prefilterMessage`)

کمتر از ۱ میلی ثانیه CPU، صفر توکن. خروجی `pass` یا `reject` با کد دلیل:

کد	شرط	امتیاز ثبت شده
<code>empty</code>	متن خالی	۵
<code>too_short</code>	کمتر از ۱۲ کاراکتر (<code>LAYER0_MIN_LENGTH</code>)	۵
<code>link_only</code>	فقط لینک، بدون متن	۵
<code>advertisement</code>	الگوی تبلیغ / عضویت / سیگنال / ارز (زیر ۱۲۰ کاراکتر)	۵
<code>injection</code>	الگوی تزریق پرامپت (<code>ignore previous instructions</code> و مشابه)	۰

سپس dedupe محتوایی ۲۴ ساعته (`isDuplicateContent`): هاش SHA-256 متن نرمال شده

(حذف نیم فاصله، یکدست سازی فاصله و حروف) با حداکثر ۲۰۰ رکورد اخیر مقایسه

می شود؛ رکورد جاری از مقایسه مستثنا است. تکراری ها با امتیاز ۵ ثبت می شوند.

Layer 1 — غربال سریع (`screenIntent`)

بولی و بدون LLM: آیا پیام سیگنال خرید، درد کاری یا تطابق کلیدواژه دارد؟

دسته ها: `buying_signal` (نشانه های خرید / قیمت / جایگزین)، `pain_signal`

(مودیان / تحریم / اکسل / قطعی)، `keyword_match` (۲≤ کلیدواژه، یا ۱ کلیدواژه در متن

بلند)، `general_question` (سؤالی و بلند). بقیه با امتیاز ۱۲ و بدون مصرف توکن

رد می شوند.

Layer 2 — تحلیل عمیق (`evaluateMessageWithLLM`)

پرامپت کامل شناسنامه محصول + ICP با کپسوله سازی XML (=)
و سلسله مراتب مرزی دستورات؛ خروجی JSON سخت گیرانه؛ `temperature: 0.1`؛
تایم اوت ۱۲ ثانیه؛ سقف هم زمانی ۳ (`llmConcurrencyLimiter`). در هر خطا،
پاسخ نامعتبر یا timeout، موتور heuristic فارسی (`heuristicPersianEvaluator`)
جایگزین می شود — همان موتوری که آفلاین کامل کار می کند. خروجی ها sanitize
می شوند (`sanitizeSuggestedReply`) : حذف تگ، خنثی سازی `javascript:` و مشابه،
سقف ۱۲۰۰ کاراکتر و `reasoning` / `matched_feature` به فارسی تضمین می شوند
(`ensurePersianText`).

حسابداری هزینه (`lib/pricing.ts`)

به ازای هر پیام: توکن ورودی / خروجی (واقعی از `usage` یا تخمین فارسی
`estimatePersianTokens` : ~۰,۶۲ توکن به ازای هر کاراکتر فارسی) ضربدر نرخ های
`INPUT/OUTPUT_TOKEN_COST_PER_MILLION` ، با معادل ریالی / تومانی برای داشبورد.
L0/L1 همیشه صفر ثبت می کنند.

6. Ingestion — جمع‌آوری پیام

وب‌هوک‌های بات (مسیر اصلی لایو)

مسیر	پلتفرم	هدر secret
POST /api/ingest/telegram	Telegram Bot API	X-Telegram-Bot-API-Secret-Token
POST /api/ingest/bale	Bale Bot API (تلگرام)	X-Bale-Secret-Token

هر دو **X-Webhook-Secret** را هم به عنوان fallback می‌پذیرند و برای push دستی /

تست، Bearer معتبر (**RADAR_API_KEY**) کافی است. ثبت وب‌هوک:

```
BASH

curl "https://api.telegram.org/bot<TELEGRAM_BOT_TOKEN>/setWebhook?url=https://<domain>/api/ingest/telegram"
# Bale: setWebhook به /api/ingest/bale با BALE_WEBHOOK_SECRET
```

پیش‌نیازها: بات ساخته‌شده، عضویت بات در سوپرگروه/کانال هدف (تلگرام: Privacy

Mode خاموش)، و HTTPS عمومی (Caddy در DEPLOY.sh تأمین می‌کند).

جریان هر درخواست (**lib/ingest.ts**):

1. احراز secret/Bearer و rate limit (۱۲۰ req/min به ازای IP).

2. استخراج پیام از **message | edited_message | channel_post** (متن یا کپشن):

آپدیت‌های غیرمتنی (استیکر، عضویت، نظرسنجی) و پیام‌ها با

{"ignored": true} تأیید و نادیده گرفته می‌شوند.

1. **findOrCreateSource**: یک ردیف **source** به ازای هر چت

(Telegram:)